Multiparty computation with secret-sharing

MPC motivation - Private Dating

Private dating

Alice and Bob meet at a bar.

- If both of them want to date together - they will find out
- If Alice doesn't want to date - she won't learn his intentions
- If Bob doesn't want to date - he won't learn her intentions

MPC motivation - Private Dating

Private dating

Alice and Bob meet at a bar.

- If both of them want to date together - they will find out
- If Alice doesn't want to date - she won't learn his intentions
- If Bob doesn't want to date - he won't learn her intentions

- Solution with *trusted third party*: Both Alice and Bob tell their intention to a trusted third party (bartender, dating app, friend)
- What if a trusted third party is not available?

MPC motivation - Private Auction

Private Auction

Many parties wish to execute a private auction

- The highest bid wins
- Only the highest bid (and bidder) is revealed

- Solution with *trusted third party*: Every bidder shares their bidding with a trusted third party (auctioneer, computer).
- What if a trusted third party is not available?

MPC motivation - Private Set Intersection

Private Set Intersection (PSI)

Intelligence agencies hold lists of potential terrorists (MI5, FBI)

- They would like to compute the intersection
- Any other information must remain secret

- The solution with a trusted third party is unacceptable. Is there any other way?

MPC motivation - Secure shuffling

Secure shuffling

- Clients: hold sensitive data.
- Shuffler: collects, shuffles, and sends client data to the analyzer.
- Analyzer: analyzes the data.

How do we protect client data if there is a small probability that the shuffler and analyzer collide?

Trusted Third Party

- All the previous challenges can be solved with the help of a trusted third party.
- Trusting a third party is a powerful assumption (in most cases, it is not available).
- Can we do this without any trusted party?

MPC - definition

MPC (Secure Multiparty Computation)

- Parties: $P_1, \ldots, P_N \in \mathcal{P}$.
- Private inputs: $x_1, \ldots, x_N$. $(x_i \rightarrow P_i)$
- Function: f (N variable)
- Goal: jointly compute $(y_1, \ldots, y_N) = f(x_1, \ldots, x_N)$ where y_i is only known to P_i (in some cases $y_1 = \ldots y_N$).

MPC - definition

MPC (Secure Multiparty Computation)

- Parties: $P_1, \ldots, P_N \in \mathcal{P}$.
- Private inputs: $x_1, \ldots, x_N$. $(x_i \rightarrow P_i)$
- Function: f (N variable)
- Goal: jointly compute $(y_1, \ldots, y_N) = f(x_1, \ldots, x_N)$ where y_i is only known to P_i (in some cases $y_1 = \ldots y_N$).

Examples

1. Private dating: Inputs: 0 (no),1 (yes). Function: $\wedge$ (logical AND)
2. Private auction: Inputs: whole number. Function: maximum
3. Terrorist Inputs: sets. Function: Intersection

Requirements

1. Correctness: parties obtain correct output (even if some parties misbehave. It is possible that the protocol halts with no outputs)
2. Privacy: Only the output is learned
3. Independence of inputs: parties cannot choose their inputs as a function of other parties' inputs
4. Fairness: if one party learns the output, then all parties learn the output

Sum

Example

The input of P_i is x_i, compute $\sum_{i=1}^{N} x_i \mod M$.

Sum

Example

The input of P_i is x_i, compute $\sum_{i=1}^{N} x_i \mod M$.

1. P_1: choose $r \in_R \mathbb{F}_M$ and send $m_1 = x_1 + r$ to P_2,
2. P_2: send $m_2 = x_2 + m_1$ to P_3;

   ...
3. P_i: send $m_i = x_i + m_{i-1}$ to P_{i+1};

   ...
4. P_N send $m_N = x_N + m_{N-1}$ to P_1
5. P_1 broadcast $y = m_N - r$.

$m_i = r + \sum_{j=1}^{i} x_j \Rightarrow y = m_N - r = \sum_{j=1}^{N} x_i$

Adversary

Adversaries (inner and outer) might attack the protocol (corrupt parties) by trying to recover private input information. Based on the attack, the parties can be:

- Honest: Follows the protocol, does not compute anything else.
- Semi-honest: Follows the protocol but tries to learn as much as possible.
- Fail stop: Semi-honest, but can halt at any moment.
- Malicious: Can deviate from the protocol in any way.

The adversary can corrupt more participants at the same time.

Attacks against SUM

- Semi-honest participants:
  - P_i (with m_{i-1}, x_i and y) unable to compute anything vulnerable.
  - P_i and P_{i+2} together:

$$m_{i+1} - m_i = \left(\sum_{j=1}^{i+1} x_j + r \right) - \left(\sum_{j=1}^{i} x_j + r \right) = x_{i+1}$$

- Fail stop: Halts the protocol.
- Malicious: Wrong result (unable to detect)

Secret sharing

Motivation

- Secret information - secret (for example, safe code, password, key, treasure map, rocket launch code,...).
- Not secure if it is stored in one device/person
- Solution - Distribute the secret into shares: The shares are stored independently, and the secret can be reconstructed if needed.

Secret sharing

Motivation

- Secret information - secret (for example, safe code, password, key, treasure map, rocket launch code,...).
- Not secure if it is stored in one device/person
- Solution - Distribute the secret into shares: The shares are stored independently, and the secret can be reconstructed if needed.
  - Bad way: Cut it into pieces, e.g. 35749134 $\rightarrow$ 35,74,91,34

Secret sharing

Motivation

- Secret information - secret (for example, safe code, password, key, treasure map, rocket launch code,...).
- Not secure if it is stored in one device/person
- Solution - Distribute the secret into shares: The shares are stored independently, and the secret can be reconstructed if needed.
  - Bad way: Cut it into pieces, e.g. 35749134 $\rightarrow$ 35,74,91,34
  - Good way: secret $s \in \mathbb{F}$ and shares $a, b, c, s - a - b - c \in \mathbb{F}$.

Shamir secret sharing (SSS)

Shamir secret sharing (k-threshold) scheme:

- P: set of participants, D : dealer
- s secret
- $A \subseteq P$ can recover the secret if $|A| \geq k$

Shamir secret sharing (SSS)

Shamir secret sharing (k-threshold) scheme:

- P: set of participants, D : dealer
- s secret
- $A \subseteq P$ can recover the secret if $|A| \geq k$

Construction (Shamir)

1. $\mathbb{F}$ *finite field (e.g.,* $\mathbb{F}_p$*) such that* $|\mathbb{F}| > |P|$. $\mathbb{F}$ *is public.*
2. *D chooses randomly* $p \in \mathbb{F}[x]$, *with* $\deg p \leq k-1$ *and* $p(0) = s$.
3. *D sends* $s_i = p(i)$ *to* P_i.

1. If $A \subseteq P, |A| \geq k$: Lagrange-interpolation
2. If $A \subseteq P, |A| < k \Rightarrow$ cannot compute anything about p

Lagrange-interpolation

Lagrange-interpolation

- $\mathbb{F}$ field, $p \in \mathbb{F}[x]$, $\deg p \leq k - 1$
- $y_i = p(x_i)$ is known for $i = 1, \ldots, k$
- Determine p

Lagrange-interpolation

Lagrange-interpolation

- $\mathbb{F}$ field, $p \in \mathbb{F}[x]$, $\deg p \leq k-1$
- $y_i = p(x_i)$ is known for $i = 1, \ldots, k$
- Determine p

1. Construct basis polynomials: $\ell_i(x_i) = 1$ and $\ell_i(x_j) = 0$, if $i \neq j$.

$$\ell_i(x) = \prod_{\substack{1 \leq j \leq k \\ j \neq i}} \frac{x - x_j}{x_i - x_j}$$

2. Construct p:

$$p(x) = \sum_{i=1}^{k} p(i)\ell_i(x)$$

Remark

To reconstruct the secret, it is enough to compute $p(0)$.

Example - Lagrange-interpolation

Example

Compute the polynomial $p \in \mathbb{F}_7$ such that $\deg(p) \leq 2$ and $p(1) = 2, p(3) = 6$ and $p(4) = 1$.

1. Compute the base polynomials:

$$\ell_1(x) = \frac{(x-3)(x-4)}{(1-3)(1-4)} = \frac{x^2 - 7x + 12}{6} = 6x^2 + 2$$

$$\ell_3(x) = \frac{(x-1)(x-4)}{(3-1)(3-4)} = \frac{x^2 - 5x + 4}{-2} = 3x^2 + 6x + 5$$

$$\ell_3(x) = \frac{(x-1)(x-3)}{(4-1)(4-3)} = \frac{x^2 - 4x + 3}{3} = 5x^2 + x + 1$$

2. Compute p:

$$2\ell_1(x) + 6\ell_3(x) + 4\ell_4(x) = 50x^2 + 40x + 38 = x^2 + 5x + 3$$

Example

Suppose that we want to generate a 3-out-of-4 secret sharing.

Secret generation

- Setup: $P = \{P_1, P_2, P_3, P_4\}$, $k = 3$, $\mathbb{F} = \mathbb{F}_7$, $s = 3$
- Polynomial: $p(x) = x^2 + 5x + 3$
- Shares: $P_1 : 2$, $P_2 : 3$, $P_3 : 6$ $P_4 : 4$

Example

Suppose that we want to generate a 3-out-of-4 secret sharing.

Secret generation

- Setup: $P = \{P_1, P_2, P_3, P_4\}$, $k = 3$, $\mathbb{F} = \mathbb{F}_7$, $s = 3$
- Polynomial: $p(x) = x^2 + 5x + 3$
- Shares: $P_1 : 2$, $P_2 : 3$, $P_3 : 6$ $P_4 : 4$

Suppose P_1, P_3, and P_4 want to reconstruct the secret from their shares.

Secret reconstruction

They collect their shares (2, 6, 4) and compute

$$2\frac{(0-3)(0-4)}{(1-3)(1-4)} + 6\frac{(0-1)(0-4)}{(3-1)(3-4)} + 4\frac{(0-1)(0-3)}{(4-1)(4-3)} =$$
$$= 2 \cdot 2 + 6 \cdot (-2) + 1 \cdot 4 = 3$$

Shamir Secret sharing - Properties

Notation

$[s]$: s is a value shared with SSS. The share of P_i is s_i.

- k-out-of-n secret sharing: SS with n parties and threshold k.
- The size of the share is $\log_2 |F|$.
- $|F| > |P|$ is important, as all participants must receive a different value of the polynomial.

Remark

Every polynomial $p \in \mathbb{F}[x], \deg p < k$, $p(0) = s$ determines a valid SSS of the secret s.

Operations with shared secrets

Lemma

Given $[a]$, $[b]$ *(SSS of a and b) and a constant c, participants can locally calculate an Shamir secret sharing of* $a + b$ *and* ca.

If the shares of P_i in $[a]$ and $[b]$ are a_i and b_i respectively, then

- $[a] + [b]$: secret sharing s.t. the shares of P_i is $a_i + b_i$.
- $c[a]$: secret sharing s.t. the shares of P_i is ca_i
- $[a] \to p$, $[b] \to q$,
- $[a] + [b]$ is a SSS of $a + b$: $r = p + q$.
  $r(i) = p(i) + q(i) = a_i + b_i \ \forall i = 0, 1, \ldots, n$.
- $c[a]$ is a SSS of ca: $r' = c \cdot p$. $r'(i) = c \cdot p(i) = c \cdot a_i$
  $\forall i = 0, 1, \ldots, n$.

SUM with SSS

Construction

$f(x_1, \ldots, x_n) = \sum_{i=1}^{n} x_i$

1. $\forall i$ *P_i shares its input as a dealer with others, sending* $p_i(j)$ *to* P_j.
2. $\forall i$ *P_i locally computes* $\sum_{j=1}^{n} p_j(i)$
3. *k parties (using Lagrange-interpolation) jointly compute* $\sum_{j=1}^{n} p_j(0)$.

Remark

- *If* $x_i = 0\ \forall i \Rightarrow$ *the parties can jointly generate* $[0]$.
- *If* x_i *is random* $\forall i \Rightarrow$, *the parties can jointly generate a shared random (unknown to them).*
- *If* $x_1 = t$ *és* $x_i = 0\ \forall i, i \neq 1 \Rightarrow$ *parties can jointly generate* $[t]$ *for any* t.

SUM with SSS example

Example

A 2-out of-3 SSS over $\mathbb{F}_{11}$.

Name	Alice (1)	Bob (2)	Cloe (3)
Input	5	2	7
Polynomial	$3x + 5$	$9x + 2$	$x + 7$
Alice's shared input	8	0	3
Bob's shared input	0	9	7
Cloe's shared input	8	9	10
Sum of shared values	5	7	9

Reconstruction of the Secret (Alice and Cloe):

$$s = 5\frac{0-3}{1-3} + 9\frac{0-1}{3-1} = 2 + 1 = 3.$$

SUM against attacks

Security:

- Semi-honest: if at most $k-1$ parties collude $\Rightarrow$ cannot compute anything vulnerable
- Halting: If at least k parties remain, they can compute SUM.
- Malicious: Suppose that every party is honest in step 1 and the number of malicious parties is at most $t \leq k-1$.
  - Attack is detectable if at least $k+t$ parties participate in the interpolation.
  - The correct result can be computed if at least $k+2t$ parties participate in the interpolation.

Multiplication

- Multiplication: Given $[a]$ and $[b]$, compute $[ab]$.
- If $r = pq$, then $r(0) = p(0)q(0) = a \cdot b$.
- P_i locally computes $a_i \dot{b}_i$.

Multiplication

- Multiplication: Given $[a]$ and $[b]$, compute $[ab]$.
- If $r = pq$, then $r(0) = p(0)q(0) = a \cdot b$.
- P_i locally computes $a_i \dot{b}_i$.

Problems

- Problem 1: The degree of r can be $2k - 2 \Rightarrow 2k - 1$ parties needed to restore the secret
- Problem 2: r not random

- Degree reduction algorithm - It takes extra communication.
- Randomize r with $[0]$.

Secure aggregation remainder

- Clients: owner of sensitive data
- Server: potentially malicious, collects and analyzes the data
- Goal: collect data without violating privacy

Secure aggregation

- Setup: The private input of client C_i is x_i.
- Mask generation: Each pair of clients generates a common mask (r_{ij} for clients C_i and C_j).
- Masking: The client C_i computes $y_i = x_i + \sum_{i<j} r_{ij} - \sum i > j r_{ij}$ and send it to the server.
- Aggregate: The server computes $s = \sum_i y_i$.

Secure aggregation properties

- Correctness: if every client is semi-honest, then the server correctly computes $\sum_i x_i$.
- Privacy: if at least two clients do not collude with the server, the server is unable to learn these client's input.

Problems

- If a client halts before sending its data to the server, the server is unable to compute the sum.
- There is no way to catch a client if it behaves maliciously.

Handling halting clients

- If a client halts, the server needs its masks to get the real answer.

Handling halting clients

- If a client halts, the server needs its masks to get the real answer.
- Idea: Every client sends its shared mask with the missing client.

Handling halting clients

- If a client halts, the server needs its masks to get the real answer.
- Idea: Every client sends its shared mask with the missing client.
- Problem 1: If some client fails to send the shared keys, then the server needs the shared masks of that client, too.
- Problem 2. It violates privacy.
  - If the client later sends y_i, then the server can learn its input.
  - A Malicious server may lie if it receives the value from the client.

Handling halting clients

- If a client halts, the server needs its masks to get the real answer.
- Idea: Every client sends its shared mask with the missing client.
- Problem 1: If some client fails to send the shared keys, then the server needs the shared masks of that client, too.
- Problem 2. It violates privacy.
  - If the client later sends y_i, then the server can learn its input.
  - A Malicious server may lie if it receives the value from the client.
- Solution:
  - Problem 1: use secret sharing.
  - Problem 2: use double-masking

Double-masking and secret sharing

Common mask via key-exchange

Key-exchange algorithm: Two parties, each holding a secret key, can generate a common key and communicate through a public channel that only two of them know.

Double-masking

The client c_i generates a random b_i and sends $z_i = y_i + b_i$ to the server.

Secret sharing

Clients use k-out-of-n secret sharing to share their masks.

Secure aggregation protocol - part 1

Secure aggregation

- Setup: The private input of the client C_i is x_i.
- Mask generation 1: Each client generates two randoms, a_i and b_i.
- Mask generation 2: Each pair of clients (C_i and C_j), using the key exchange protocol for a_i and a_j, generates a mask pair, r_{ij} in C_i and $r_{ji} = -r_{ij}$) in C_j.
- Share: Each client C_i shares a_i and b_i with other clients via k-out-of-n secret sharing.
- Masking: The client C_i computes $z_i = x_i + b_i + \sum_j r_{ij}$ and sends it to the server.

Secure aggregation protocol - part 2

Secure aggregation

- Collecting a personal mask from survivors ($\mathcal{S}$): The server sends $C_i \in \mathcal{S}$ the list of surviving clients. For every client C_j,
  - if $C_j \in \mathcal{S}$, then C_i sends the share of b_j.
  - if $C_j \notin \mathcal{S}$, then C_i sends the share of a_j.
- Reconstruction: The server reconstructs a_i and b_i for every $C_i \in \mathcal{S}$, and computes r_{ij} from a_i and a_j for every i, j.
- Aggregate: The server computes $s = \sum_{i \in \mathcal{S}} z_i - \sum_{i \in \mathcal{S}} b_i - \sum_{i \in \mathcal{S}, j \notin \mathcal{S}} r_{ij}$.

Properties

Correctness

$\sum_{i \in \mathcal{S}} z_i = \sum_{i \in \mathcal{S}} x_i + \sum_{i \in \mathcal{S}} b_i + \sum_{i \in \mathcal{S}, j \notin \mathcal{S}} r_{ij}$, because $\sum_{i \in \mathcal{S}, j \notin \mathcal{S}} r_{ij}$ is the part that does not cancel out. Therefore, the server successfully computes $z = \sum_{i \in \mathcal{S}} x_i$.

Output delivery

If at least k clients survive, then the server receives at least k share of each mask to recover.

Security

Semi-honest adversary:

- Clients only: no information on private input.
- Server + at most $k-1$ clients: if there are at least two surviving non-colluding clients C_i and C_j, it is secure because the colluders have no information on r_{ij}.
- Server + at least k clients: not secure because they can reconstruct a_i and b_i for every client.

Malicious adversary:

- Clients only: Still secure. More tools are needed to achieve correctness.
- Server: Can lie about survivors and send different sets to each client, resulting in learning a_i and b_i for every client. Using a digital signature can solve this problem.